

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ОДЕСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ І.І.МЕЧНИКОВА
Факультет математики, фізики та інформаційних технологій
Кафедра математичного забезпечення комп'ютерних систем

КУРСОВА РОБОТА
з освітнього компоненту «Програмування»
на тему: СТВОРЕННЯ ІЄРАРХІЇ КЛАСІВ НА ТЕМУ «МУЗЕЙ»

Студента 1 курсу, денна форма навчання
спеціальність F7 «Комп'ютерна інженерія»

Напалкова Кирила Олександровича

Керівник: к.ф.-м.н., доц. Антоненко О.С.

Національна шкала: _____

Кількість балів: _____

ECTS: _____

Дата захисту: _____

Члени комісії _____ Антоненко О.С.

(підпис) (прізвище та ініціали)

_____ Шестопалов С.В.

(підпис) (прізвище та ініціали)

(підпис) (прізвище та ініціали)

ЗМІСТ

	стор.
ВСТУП.....	3
ЗАВДАННЯ.....	4
1. ТЕОРЕТИЧНА ЧАСТИНА.....	5
1.1 Поняття ООП.....	5
1.2 Опис об'єктної частини	6
2. ПРАКТИЧНА ЧАСТИНА.....	8
2.1 Опис класів	8
2.2 Інтерфейс	12
ВИСНОВКИ	15
СПИСОК ЛІТЕРАТУРИ.....	16

ВСТУП

Дана курсова робота присвячена проектуванню та реалізації ієрархії класів на тему «Музей» засобами мови програмування C++. Головна мета полягає у практичному застосуванні концепцій об'єктно-орієнтованого програмування для моделювання реальної предметної області.

У рамках роботи необхідно спроектувати взаємопов'язану систему класів, що відображає структуру музею: кімнати як фізичний простір та різноманітні типи експонатів, що у них розміщуються. Ключовою вимогою є перевірка коректності даних на всіх рівнях – від розмірів кімнати до наявності вільного місця під час розстановки експонатів.

При виконанні роботи використовуються три фундаментальні принципи ООП. Інкапсуляція забезпечує приховання внутрішніх даних класів та надає доступ до них лише через публічні методи. Наслідування дозволяє будувати ієрархію від загального абстрактного класу Експонат до конкретних типів – Картина, Скульптура, Техніка, Предмет побуту. Поліморфізм через віртуальні методи забезпечує єдиний інтерфейс для роботи з будь-яким типом експонату.

Програму реалізовано мовою C++ з розміщенням кожного класу в окремому файлі у папці `classes`, що відповідає сучасним стандартам організації коду.

ЗАВДАННЯ

Варіант 8а. Створення ієрархії класів на тему «Музей»

Створити клас кімната, що задає розміри кімнати: ширина, довжина, висота, корисна площа стін (перевірити, щоб корисна площа стін була не більша від загальної площі). Створити клас "Експонат" (автор, країна, рік, врахувати, що автор може бути невідомий), зі спадкоємцями "Картина" (розміри: ширина, висота), "Об'ємний експонат" (ширина, довжина, висота, необхідна для розміщення експонату), у об'ємного експонату спадкоємці "Скульптура", "Техніка", "Предмет побуту".

Реалізувати розміщення експонатів по кімнатах (перевіряючи, чи експонат розміщується в кімнаті).

1. ТЕОРЕТИЧНА ЧАСТИНА

1.1 Поняття ООП

Об'єктно-орієнтоване програмування (ООП) – це парадигма програмування, в якій програма розглядається як сукупність об'єктів, що взаємодіють між собою. Кожен об'єкт є екземпляром певного класу, а класи можуть утворювати ієрархії наслідування. Програміст спочатку описує клас – шаблон даних і поведінки – а вже під час виконання програми на основі класу створюються конкретні об'єкти.

ООП базується на трьох ключових концепціях:

- а) інкапсуляція;
- б) наслідування;
- с) поліморфізм.

Інкапсуляція – це механізм об'єднання даних і методів, що працюють з цими даними, в єдину сутність – клас. Водночас вона приховує внутрішню реалізацію від зовнішнього коду, надаючи доступ лише через публічний інтерфейс. Це захищає дані від некоректного використання та спрощує супровід програми.

Наслідування – це механізм, що дозволяє новому класу (нащадку) перебирати поля і методи вже існуючого класу (батька), розширюючи або змінюючи його поведінку. Завдяки наслідуванню усувається дублювання коду і будується чітка ієрархія понять предметної області.

Поліморфізм дає змогу звертатися до об'єктів різних класів через єдиний інтерфейс. У C++ він реалізується за допомогою віртуальних методів: виклик через вказівник або посилання на базовий клас автоматично направляється до реалізації в конкретному нащадку під час виконання програми.

1.2 Опис об'єктної частини

Розроблена система моделює роботу музею та складається з двох незалежних гілок класів: Room, що описує фізичний простір, та ієрархії Exhibit, що представляє експонати різних типів.

Клас Exhibit є абстрактним базовим класом для всіх експонатів. Він зберігає загальні атрибути: назву, автора, країну та рік. Передбачено особливий випадок – якщо автор невідомий, замість порожнього рядка автоматично підставляється константа UNKNOWN_AUTHOR зі значенням «Невідомий». Клас оголошує два чисто віртуальних методи: fitsInRoom для перевірки можливості розміщення та getType для отримання рядкового опису типу. Метод printInfo є віртуальним і виводить базову інформацію, яку кожен нащадок доповнює своїми полями.

Клас Painting є прямим нащадком Exhibit і моделює картину з двома розмірами – шириною та висотою. Оскільки картина вішається на стіну, метод fitsInRoom перевіряє не лише габарити відносно стін кімнати, а й те, чи вистачає корисної площі стін для її розміщення. Метод getWallArea повертає площу картини для обліку зайнятого місця на стінах.

Клас VolumetricExhibit є проміжним абстрактним нащадком Exhibit, що додає три розміри для об'ємних об'єктів: ширину, довжину та висоту. Метод fitsInRoom перевіряє, чи вміщується об'єкт за всіма трьома вимірами кімнати. Метод getFootprint повертає площу підлоги, яку займає експонат. Від VolumetricExhibit успадковуються три конкретні класи:

- a) Sculpture – скульптура, додає поле матеріалу (мармур, бронза тощо);
- b) Technics – технічний експонат, зберігає тип пристрою та ознаку діючого стану;
- c) HouseholdItem – предмет побуту, містить категорію та епоху виготовлення.

Клас `Room` описує кімнату музею. У конструкторі виконується перевірка: корисна площа стін не може перевищувати загальну площу $2 * (\text{ширина} + \text{довжина}) * \text{висота}$. При додаванні нового експонату методом `addExhibit` кімната послідовно перевіряє три умови: відповідність габаритів, наявність вільної площі підлоги для об'ємного експоната та наявність вільної площі стін для картини. Кімната зберігає список розміщених експонатів у вигляді вектора розумних вказівників `shared_ptr<Exhibit>`, що автоматично керує пам'яттю.

2. ПРАКТИЧНА ЧАСТИНА

2.1 Опис класів

Діаграму класів наведено в додатку А. Нижче детально описано кожен клас проекту.

1) Абстрактний клас «Exhibit»:

- a) Захищені поля: title, author, country, year.
- b) Статична константа: UNKNOWN_AUTHOR = «Невідомий».
- c) Конструктор з параметрами (назва, автор, країна, рік). Якщо рядок автора порожній – підставляється UNKNOWN_AUTHOR.
- d) Методи:
 - getTitle, getAuthor, getCountry, getYear – геттери відповідних полів.
 - isAuthorKnown – повертає true якщо автор відомий.
 - getFootprint – віртуальний; за замовчуванням повертає 0; перевизначається у VolumetricExhibit.
 - getWallArea – віртуальний; за замовчуванням повертає 0; перевизначається у Painting.
 - fitsInRoom(const Room&) – чисто віртуальний; реалізується у кожному нащадку.
 - getType – чисто віртуальний; повертає рядок з назвою типу.
 - printInfo – віртуальний; виводить загальну інформацію про експонат.

2) Клас «Painting», нащадок «Exhibit»:

- a) Приватні поля: paintWidth, paintHeight – ширина та висота картини в метрах.
- b) Конструктор з параметрами (назва, автор, країна, рік, ширина, висота).
- c) Методи:

- `getPaintWidth`, `getPaintHeight` – геттери розмірів.
- `getArea` – повертає площу картини (ширина $\times$ висота).
- `getWallArea` – перевизначення; повертає площу для обліку зайнятого місця на стінах.
- `fitsInRoom` – перевіряє: ширина $\leq$ ширина кімнати, висота $\leq$ висота кімнати, площа $\leq$ корисна площа стін.
- `getType` – повертає рядок «Картина».
- `printInfo` – перевизначення; додатково виводить розміри та площу.

3) Клас «`VolumetricExhibit`», нащадок «`Exhibit`»:

- a) Захищені поля: `objWidth`, `objLength`, `objHeight` – три розміри об'ємного експонату.
- b) Конструктор з параметрами (назва, автор, країна, рік, ширина, довжина, висота).
- c) Методи:
 - `getObjWidth`, `getObjLength`, `getObjHeight` – геттери розмірів.
 - `getFootprint` – перевизначення; повертає площу підлоги (ширина $\times$ довжина).
 - `fitsInRoom` – перевіряє всі три розміри відносно відповідних розмірів кімнати.
 - `getType` – повертає рядок «Об'ємний експонат».
 - `printInfo` – перевизначення; виводить усі три розміри.

4) Клас «`Sculpture`», нащадок «`VolumetricExhibit`»:

- a) Приватне поле: `material` – матеріал виготовлення (наприклад, мармур, бронза).
- b) Конструктор з параметрами (назва, автор, країна, рік, ширина, довжина, висота, матеріал).
- c) Методи:

- `getMaterial` – геттер матеріалу.
- `getType` – повертає рядок «Скульптура».
- `printInfo` – перевизначення; додатково виводить матеріал.

5) Клас «Technics», нащадок «VolumetricExhibit»:

- a) Приватні поля: `techType` – тип технічного пристрою; `isWorking` – ознака діючого стану.
- b) Конструктор з параметрами (назва, автор, країна, рік, ширина, довжина, висота, тип, діючий).
- c) Методи:
 - `getTechType`, `getIsWorking` – геттери відповідних полів.
 - `getType` – повертає рядок «Техніка».
 - `printInfo` – перевизначення; виводить тип та стан пристрою.

6) Клас «HouseholdItem», нащадок «VolumetricExhibit»:

- a) Приватні поля: `itemCategory` – категорія предмету (меблі, посуд тощо); `era` – епоха або культура.
- b) Конструктор з параметрами (назва, автор, країна, рік, ширина, довжина, висота, категорія, епоха).
- c) Методи:
 - `getItemCategory`, `getEra` – геттери відповідних полів.
 - `getType` – повертає рядок «Предмет побуту».
 - `printInfo` – перевизначення; виводить категорію та епоху.

7) Клас «Room»:

- a) Приватні поля: `name`, `width`, `length`, `height`, `usableWallArea`, `exhibits` – вектор розумних вказівників на експонати.
- b) Конструктор з параметрами (назва, ширина, довжина, висота, корисна площа стін). Якщо корисна площа стін перевищує загальну площу $2 * (\text{ширина} + \text{довжина}) * \text{висота}$ – генерується виключення.

с) Методи:

- getName, getWidth, getLength, getHeight, getUsableWallArea – геттери полів.
- getTotalWallArea – обчислює та повертає загальну площу стін кімнати.
- getFloorArea – повертає площу підлоги (ширина × довжина).
- getUsedFloorArea – підраховує площу підлоги, зайняту усіма об'ємними експонатами.
- getUsedWallArea – підраховує площу стін, зайняту усіма картинами.
- addExhibit – перевіряє габарити та наявність вільного місця; додає експонат у список або виводить причину відмови.
- printInfo – виводить усі параметри кімнати та залишок вільної площі.
- printExhibits – виводить список усіх розміщених експонатів із деталями.

2.2 Інтерфейс

Готовий проєкт розміщено за посиланням [4]. При запуску програми користувач потрапляє до головного меню (Рис. 2.1), яке циклічно повторюється до введення нуля.

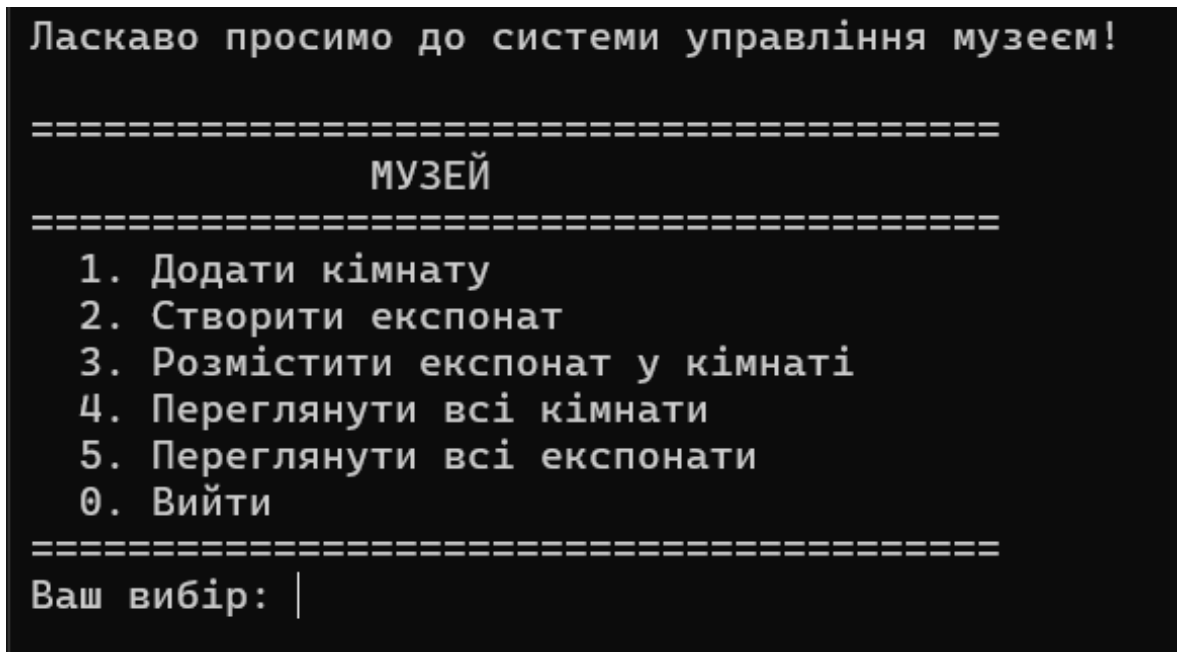


Рисунок 2.1 – Головне меню програми

При виборі пункту 1 (Рис. 2.2) програма послідовно запитує назву кімнати, ширину, довжину та висоту. Після введення розмірів автоматично розраховується та відображається загальна площа стін, після чого пропонується ввести корисну площу. Якщо введені значення перевищують загальне — виводиться повідомлення про помилку і кімната не створюється.

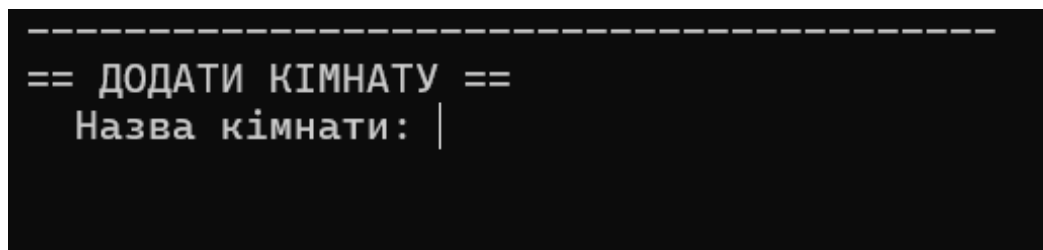


Рисунок 2.2 – Вікно додавання кімнати

При виборі пункту 2 (Рис. 2.3) спочатку відображається меню вибору типу: Картина, Скульптура, Техніка або Предмет побуту. Потім вводяться спільні для всіх типів поля – назва, автор (поле можна залишити порожнім для невідомого автора), країна та рік. Після цього вводяться специфічні характеристики обраного типу.

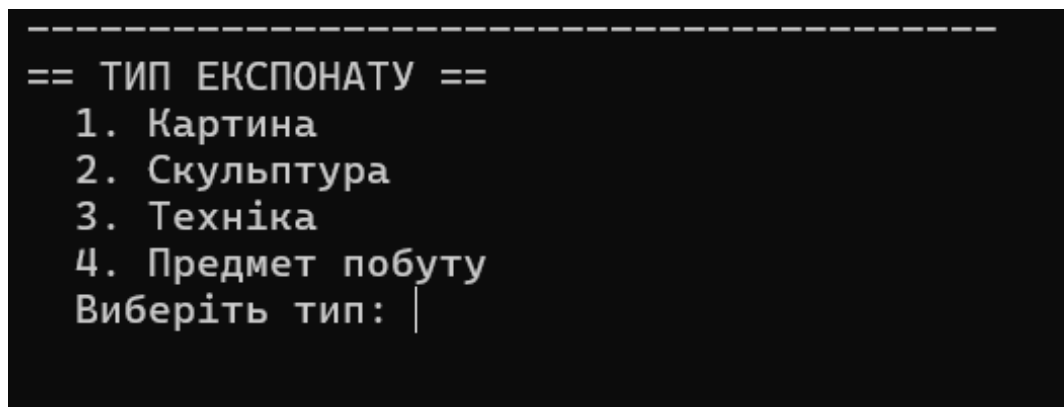


Рисунок 2.3 – Вікно створення експоната

При виборі пункту 3 (Рис. 2.4) відображаються два пронумерованих списки: всіх наявних експонатів та всіх кімнат. Після вибору конкретного експоната та кімнати програма виконує поетапну перевірку:

1. відповідність габаритів експоната розмірам кімнати;
2. достатність вільної площі підлоги для об'ємного експоната;
3. достатність вільної площі стін для картини.

За успішної перевірки експонат додається до кімнати. У разі невдачі виводиться детальне повідомлення із зазначенням наявного вільного місця та необхідного.

```

=====
== ВИБІР ЕКСПОНАТУ ==
  1. [Картина] 1
  Номер експонату: |

```

Рисунок 2.4 – Вікно розміщення експоната

При виборі пункту 4 (Рис. 2.5) для кожної кімнати послідовно виводиться: назва, розміри, площа підлоги, загальна та корисна площа стін, поточне залишкове вільне місце, а також повний перелік розміщених експонатів із їхніми характеристиками.

```

=====
== УСІ КІМНАТИ ==
=== Кімната: 1 ===
  Ширина:  2.00 м
  Довжина: 2.00 м
  Висота:   3.00 м
  Площа підлоги:           4.00 м²
  Загальна площа стін:     24.00 м²
  Корисна площа стін:      3.00 м²
  Кількість експонатів:    1
--- Експонати кімнати "1" ---
  1. [Картина] 1
    Автор: 1
    Країна: 1
    Рік: 1
    Розмір: 1.00 x 1.00 м (площа: 1.00 м²)

```

Рисунок 2.5 – Перегляд усіх кімнат

При виборі пункту 5 виводиться нумерований список усіх створених експонатів з типом, назвою, автором, країною, роком та специфічними полями кожного об'єкта.

ВИСНОВКИ

У результаті виконання курсової роботи розроблено програму мовою C++, що реалізує ієрархію класів для моделювання роботи музею. Створена система охоплює кімнати як фізичний простір та чотири типи музейних експонатів — картини, скульптури, технічні пристрої та предмети побуту.

У процесі розробки повною мірою застосовано три принципи ООП. Інкапсуляція реалізована через приватні та захищені поля з доступом виключно через публічні методи. Наслідування утворює дворівневу ієрархію: абстрактний Exhibit породжує Painting і VolumetricExhibit, який у свою чергу є батьком для Sculpture, Technics та HouseholdItem. Поліморфізм забезпечується чисто віртуальними методами fitsInRoom і getType, що дають змогу працювати з будь-яким типом експоната через єдиний інтерфейс базового класу.

Реалізовано багаторівневу перевірку даних: у конструкторі Room контролюється коректність площі стін; при розміщенні методом addExhibit перевіряються як геометричні розміри, так і залишок вільного місця на підлозі чи стінах. Для безпечного управління пам'яттю використано розумні вказівники shared_ptr, що виключають витоки пам'яті.

Практична цінність роботи полягає в закріпленні навичок проектування ієрархій класів, роботи з абстрактними класами, віртуальними методами, шаблонними контейнерами та консольним інтерфейсом мовою C++.

СПИСОК ЛІТЕРАТУРИ

1. Web Library, Основні поняття ООП. [Електронне видання] / Тернопіль 2026. – Режим доступу: <http://weblib.com.ua/blog/osnovni-ponyattya-oor>
2. W3School, C++ OOP (Object-Oriented Programming). Режим доступу: https://www.w3schools.com/cpp/cpp_oop.asp
3. Є.О. Віктор, О.А. Геренко, І.М. Шпінарева, Практикум з курсу Об'єктно-орієнтоване програмування мовою C++. [Електронне видання] / Одеса, 2026.
4. Репозиторій GitHub – Режим доступу: <https://github.com/kiruha-why-not/Kursova.Museum.Napalkov>

Додаток А. Діаграма класів

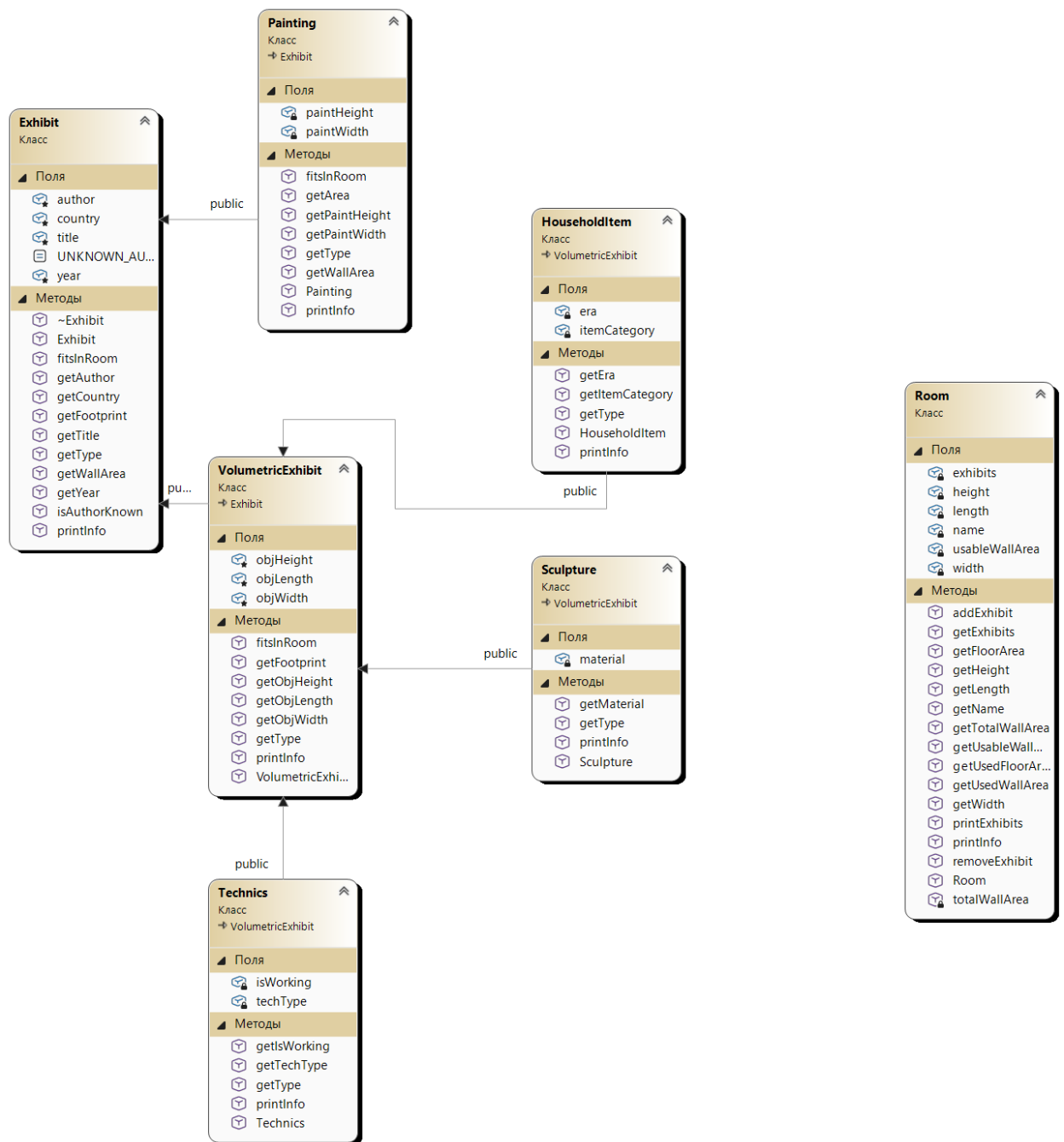


Рисунок А1. – Діаграма класів

Додаток Б. Код класів

```

#include <iostream>
#include "Exhibit.h"
#include "Room.h"
#include <stdexcept> const
std::string Exhibit::UNKNOWN_AUTHOR = "Невідомий";
Exhibit::Exhibit(const std::string& title,
                 const std::string& author,
                 const std::string& country,
                 int year)
    : title(title),
    author(author.empty() ? UNKNOWN_AUTHOR : author),
    country(country),
    year(year)
{
    if (title.empty())
        throw std::invalid_argument("Назва експонату не може бути порожньою.");
}
std::string Exhibit::getTitle() const { return title; }
std::string Exhibit::getAuthor() const { return author; }
std::string Exhibit::getCountry() const { return country; }
int Exhibit::getYear() const { return year; }

bool Exhibit::isAuthorKnown() const {
    return author != UNKNOWN_AUTHOR;
}

void Exhibit::printInfo() const {
    std::cout << "[" << getType() << "]" " << title << "\n"
    << " Автор: " << author << "\n"
    << " Країна: " << country << "\n"
    << " Рік: " << year << "\n";
}

```

Лістинг Б.1 – Клас Exhibit (Exhibit.cpp)

```

#include "VolumetricExhibit.h"
#include "Room.h"
#include <iostream>
#include <stdexcept>
#include <iomanip>

VolumetricExhibit::VolumetricExhibit(const std::string& title,
                                       const std::string& author,
                                       const std::string& country,
                                       int year,
                                       double width,
                                       double length,
                                       double height)
    : Exhibit(title, author, country, year),
    objWidth(width), objLength(length), objHeight(height)
{
    if (width <= 0 || length <= 0 || height <= 0)
        throw std::invalid_argument("Розміри об'єкта експонату мають бути додатними.");
}

double VolumetricExhibit::getObjWidth() const { return objWidth; }
double VolumetricExhibit::getObjLength() const { return objLength; }
double VolumetricExhibit::getObjHeight() const { return objHeight; }

```

Лістинг Б.2 – Клас VolumetricExhibit, аркуш 1

```

bool VolumetricExhibit::fitsInRoom(const Room& room) const {
    // Перевірка: об'єкт поміщається за шириною, довжиною та висотою
    return (objWidth <= room.getWidth()) &&
           (objLength <= room.getLength()) &&
           (objHeight <= room.getHeight());
}

std::string VolumetricExhibit::getType() const { return "Об'ємний експонат"; }
double VolumetricExhibit::getFootprint() const { return objWidth * objLength; }

void VolumetricExhibit::printInfo() const {
    Exhibit::printInfo();
    std::cout << std::fixed << std::setprecision(2)
               << "    Розмір: " << objWidth << " x " << objLength
               << " x " << objHeight << " м\n";
}

```

Лістинг Б.2 – Клас VolumetricExhibit, аркуш 2

```

#include "Painting.h"
#include "Room.h"
#include <iostream>
#include <stdexcept>
#include <iomanip>
Painting::Painting(const std::string& title,
                  const std::string& author,
                  const std::string& country,
                  int year,
                  double width,
                  double height)
    : Exhibit(title, author, country, year),
      paintWidth(width),
      paintHeight(height)
{
    if (width <= 0 || height <= 0)
        throw std::invalid_argument("Розміри картини мають бути додатними.");
}

double Painting::getPaintWidth() const { return paintWidth; }
double Painting::getPaintHeight() const { return paintHeight; }
double Painting::getArea() const { return paintWidth * paintHeight; }

bool Painting::fitsInRoom(const Room& room) const {
    // Картина вміщується, якщо вона вузька за ширину кімнати,
    // не вища за стіну, і площа <= корисна площа стін
    return (paintWidth <= room.getWidth()) &&
           (paintHeight <= room.getHeight()) &&
           (getArea() <= room.getUsableWallArea());
}

std::string Painting::getType() const { return "Картина"; }
double Painting::getWallArea() const { return getArea(); }

void Painting::printInfo() const {
    Exhibit::printInfo();
    std::cout << std::fixed << std::setprecision(2)
               << "    Розмір: " << paintWidth << " x " << paintHeight
               << " м (площа: " << getArea() << " м²)\n";
}

```

Лістинг Б.3 – Клас Painting

```

#include "HouseholdItem.h"
#include <iostream>

HouseholdItem::HouseholdItem(const std::string& title,
                             const std::string& author,
                             const std::string& country,
                             int year,
                             double width,
                             double length,
                             double height,
                             const std::string& itemCategory,
                             const std::string& era)
    : VolumetricExhibit(title, author, country, year, width, length, height),
      itemCategory(itemCategory),
      era(era)
{}

std::string HouseholdItem::getItemCategory() const { return itemCategory; }
std::string HouseholdItem::getEra() const { return era; }

std::string HouseholdItem::getType() const { return "Предмет побуту"; }

void HouseholdItem::printInfo() const {
    VolumetricExhibit::printInfo();
    std::cout << " Категорія: " << itemCategory << "\n"
    << " Епоха: " << era << "\n";
}

```

Лістинг Б.4 – Клас HouseholdItem

```

#include "Sculpture.h"
#include <iostream>
Sculpture::Sculpture(const std::string& title,
                    const std::string& author,
                    const std::string& country,
                    int year,
                    double width,
                    double length,
                    double height,
                    const std::string& material)
    : VolumetricExhibit(title, author, country, year, width, length, height),
      material(material)
{}

std::string Sculpture::getMaterial() const { return material; }

std::string Sculpture::getType() const { return "Скульптура"; }

void Sculpture::printInfo() const {
    VolumetricExhibit::printInfo();
    std::cout << " Матеріал: " << material << "\n";
}

```

Лістинг Б.5 – Клас Sculpture

```

#include "Technics.h"
#include <iostream>
Technics::Technics(const std::string& title,
                  const std::string& author,
                  const std::string& country,
                  int year,
                  double width,
                  double length,
                  double height,
                  const std::string& techType,
                  bool isWorking)
    : VolumetricExhibit(title, author, country, year, width, length, height),
      techType(techType),
      isWorking(isWorking)
{}

std::string Technics::getTechType() const { return techType; }
bool Technics::getIsWorking() const { return isWorking; }

std::string Technics::getType() const { return "Техніка"; }

void Technics::printInfo() const {
    VolumetricExhibit::printInfo();
    std::cout << " Тип техніки: " << techType << "\n"
    << " Діючий: " << (isWorking ? "Так" : "Ні") << "\n";
}

```

Лістинг Б.6 – Клас Technics

```

#include "Room.h"
#include "Exhibit.h"
#include <iostream>
#include <stdexcept>
#include <iomanip>

double Room::totalWallArea() const {
    return 2.0 * (width + length) * height;
}

Room::Room(const std::string& name,
           double width,
           double length,
           double height,
           double usableWallArea)
    : name(name), width(width), length(length), height(height),
      usableWallArea(usableWallArea)
{
    if (width <= 0 || length <= 0 || height <= 0)
        throw std::invalid_argument("Розміри кімнати мають бути додатними.");

    double total = totalWallArea();
    if (usableWallArea < 0 || usableWallArea > total) {
        throw std::invalid_argument(
            "Корисна площа стін (" + std::to_string(usableWallArea) +
            " м²) не може перевищувати загальну площу стін (" +
            std::to_string(total) + " м²).");
    }
}

```

Лістинг Б.7 – Клас Room, аркуш 1

```

std::string Room::getName() const { return name; }
double Room::getWidth() const { return width; }
double Room::getLength() const { return length; }
double Room::getHeight() const { return height; }
double Room::getUsableWallArea() const { return usableWallArea; }
double Room::getTotalWallArea() const { return totalWallArea(); }
double Room::getFloorArea() const { return width * length; }

double Room::getUsedFloorArea() const {
    double used = 0;
    for (const auto& ex : exhibits)
        used += ex->getFootprint();
    return used;
}

double Room::getUsedWallArea() const {
    double used = 0;
    for (const auto& ex : exhibits)
        used += ex->getWallArea();
    return used;
}

bool Room::addExhibit(const std::shared_ptr<Exhibit>& exhibit) {
    if (!exhibit->fitsInRoom(*this)) {
        std::cout << "[!] Експонат \"" << exhibit->getTitle()
            << "\" не поміщається за розмірами.\n";
        return false;
    }

    double freeFloor = getFloorArea() - getUsedFloorArea();
    double freeWall = getUsableWallArea() - getUsedWallArea();

    if (exhibit->getFootprint() > 0 && exhibit->getFootprint() > freeFloor) {
        std::cout << "[!] Недостатньо місця на підлозі. "
            << "Вільно: " << freeFloor
            << " м2, потрібно: " << exhibit->getFootprint() << " м2\n";
        return false;
    }

    if (exhibit->getWallArea() > 0 && exhibit->getWallArea() > freeWall) {
        std::cout << "[!] Недостатньо місця на стінах. "
            << "Вільно: " << freeWall
            << " м2, потрібно: " << exhibit->getWallArea() << " м2\n";
        return false;
    }

    exhibits.push_back(exhibit);
    std::cout << "[+] Експонат \"" << exhibit->getTitle() << "\" розміщено!\n";
    return true;
}

void Room::removeExhibit(int index) {
    if (index < 0 || index >= (int)exhibits.size())
        throw std::out_of_range("Невірний індекс експонату.");
    exhibits.erase(exhibits.begin() + index);
}

const std::vector<std::shared_ptr<Exhibit>>& Room::getExhibits() const {
    return exhibits;
}

```

Лістинг Б.7 – Клас Room, аркуш 2

```

void Room::printInfo() const {
    std::cout << std::fixed << std::setprecision(2);
    std::cout << "=== Кімната: " << name << " ===\n"
        << " Ширина: " << width << " м\n"
        << " Довжина: " << length << " м\n"
        << " Висота: " << height << " м\n"
        << " Площа підлоги: " << getFloorArea() << " м²\n"
        << " Загальна площа стін: " << getTotalWallArea() << " м²\n"

        << " Корисна площа стін: " << usableWallArea << " м²\n"
        << " Кількість експонатів: " << exhibits.size() << "\n";
}
void Room::printExhibits() const {
    std::cout << "---- Експонати кімнати \"" << name << "\" ----\n";
    if (exhibits.empty()) {
        std::cout << " (порожньо)\n";
        return;
    }
    for (size_t i = 0; i < exhibits.size(); ++i) {
        std::cout << " " << (i + 1) << ". ";
        exhibits[i]->printInfo();
    }
}

```

Лістинг Б.7 – Клас Room, аркуш 3